

Unit-3

Section A

Q1) What is the purpose of the try and catch blocks in Java?

Ans) try - This block contains the code that might throw an exception. The code in the try block is executed normally until an exception occurs.

Catch - Contains code that handles the exception if it occurs in the try block.

Q2) Define an exception in Java.

Ans) An exception is an unwanted event or unexpected event which occurs during the execution of the program, that disturbs the normal flow of the program.

Q3) What is the role of the "finally" block in exception handling?

Ans) The finally block is typically used for clean-up code make sure the file always close after it is used even if an exception was thrown in the try block.

Q4) What is the "throw" keyword and how is it used in exception handling?

Ans) Throw keyword does not use for pre-defined Exception.

2) Throw always use user-defined/customized exception.

3) Throw used always (unchecked Exception/Run-time Exception).

Syntax:-

throw new Exception className ("String msg")

Q5) Define the Thread class in Java.

Ans) The Thread class is the main class that supports the multithreading system. It provides constructs and methods for creating and operating on threads.

Syntax:-

```
class Thread
```

```
{
```

```
    // constructor
```

```
    // method
```

```
    1) run
```

```
    2) start
```

```
    3) sleeping
```

```
    4) waiting
```

```
    5) set priority / get priority.
```

```
}
```

Q6) Explain Throws?

Ans) Throws keyword is used to declare an exception. It gives an information to the caller method that there may occur an Exception, so it is better for the caller method to provide the Exception Handling code so the normal flow can be maintained.

Q7) Differentiate b/w the Thread class and the Runnable interface in Java.

Ans)

Thread class

1) Thread class is a class that provides a way to create and manage threads.

2) It has multiple methods include (start(), run(),).

3) Each thread executes a unique object and gets associated with it.

4) More memory required

5) Thread class create a new Thread directly.

Runnable Interface
Provides task to be executed by a thread.

It has only abstract method run().

multiple threads share the same objects.

less memory required

Does not create a new Thread directly.

Q8) Difference b/w "Error" and "Exception" in the context of exception handling.

Ans)

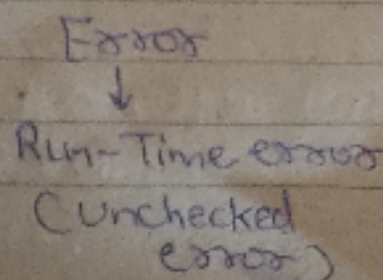
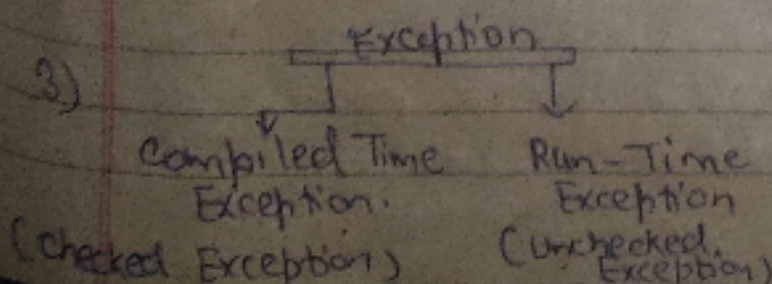
Error

1) Lack of System resources
Eg:- Processor (slow-etc).

2) Recoverable by using Exception Handling Techniques such as:-
(try, catch, finally, throw throws).

Exception
Program

Error are not recoverable
programmer can handle them to their level.



Q9) Explain the Rules of Thread class?

Ans)

- Rule 1) Extends the Thread class.
- 2) Overriding the run method.
- 3) Create an object of the class.
- 4) Start the thread.

Q10) Explain the Rules for Runnable Interface?

Ans)

- Rule 1) Implements the Runnable Interface.
- 2) Overriding the run method.
- 3) Create an object of the class.
- 4) Create an object of Thread class & pass the parameter in constructor.
- 5) Start the Thread.

Q11) What does the start () method of a thread do?

Ans)

The start () method of a thread in Java is crucial for initiating a new thread of execution.

Q12) Define Thread priority in Java?

Ans)

Java provide the priority to each thread and according to these priorities JVM will allocate the processors.

Priorities are represented in the form of integer value i.e. range from 1-10.

1 $\Rightarrow$ MIN-Priority.

5 $\Rightarrow$ NORM-Priority.

10 $\Rightarrow$ MAX-Priority.

10 9 8 7 6 5 4 3 2 1

1-10 $\Rightarrow$ 0%.

More priority will execute fast
Less priority will execute slow.

Q.3) Can the finally block be skipped during exception?
Explain.

Ans) No, the finally block cannot be skipped during execution. In Java the finally block is designed to execute after the try block regardless of whether an exception was thrown or not.

Q.4) What is multi threading in Java?

Ans) Multithreading in Java is a programming concept that allows multiple threads (or lightweight processes) to run concurrently within a single Java program. Multithreading enables a Java program to perform multiple tasks simultaneously.

Types of multithreading in Java are:-

- 1) Multi-Processing
- 2) Multi-Threading.

Q.5) How do you set thread priority in Java?

Ans)

Q.6) What is the use of Synchronized keyword in Java?

Ans) The use of Synchronized keyword in Java when multiple threads access and modify a shared variable, synchronized is essential to avoid inconsistencies.

Q17) What is the role of `System.out.println(e.toString())` in a catch block?

Ans) The role of `System.out.println(e.toString())` in a catch block is to print a concise description of an exception that occurred.

eg:-

try

{
int result = 10/0;

}

catch (ArithmeticException e)

{

System.out.println(e.toString());
}

// output: java.lang.ArithmeticException:

Q18) How to print Exception Handling?

Ans)

- 1) Exception Name
- 2) Description.
- 3) Stack Trace.

i) `e.printStackTrace();`

EN(✓)

DS(✓)

ST(✓)

ii) `SOP(e);`

EN(✓)

DS(✓)

ST(X)

iv) `SOP(e.getMessage());`

EN(X)

DS(✓)

ST(X)

iii) `SOP(e.toString());`

EN(✓)

DS(✓)

ST(X)

Q19) Can we have multiple finally blocks for a single try block? Explain.

Ans) No, we cannot have multiple finally blocks for a single try block in Java because according to the syntax the finally block allowed only one.

Q20) Give Three Rules for Finally block?

Ans) 1) We can use multiple catch blocks with one try block but we can use single finally block with one try block.

2) The statement present in the finally block execute even if the try block contain control transfer statement (Jump statement) eg:- return, break, switch.

3) The possibilities that disturb the execution of finally block are:-

a) it use of `System.exit()` mtd.

b) it causing a fatal error that causes the process to abort.

Section - B

Q21) Explain atleast six differences b/w the throw and throws keywords.

Ans) throw keyword
The throw keyword is used to explicitly throw an exception from within a block of code or a method.

throws keyword
The throws keyword is used to declare the exception.

i) The throw keyword can only throw a single exception at a time.

The throws keyword allows for the declaration of multiple exceptions.

ii) The throw keyword used on always unchecked Exceptions.

The throws keyword used to declare both checked and unchecked exceptions.

iii) Syntax:-
throw new Exception class Name ("String msg");

iv) Essential for handling exceptions in try-catch blocks.

Provides information to callers about potential exceptions.

v) Used within a method or block or raise an exception.

Used in a method declaration to indicate exceptions.

Q22) Write a program in Java to implement Exception Handling?

Ans) Class Main

```
{  
    public static void main (String[] args)  
    {  
        try  
        {  
            int a = 100, b = 0, c;  
            c = a/b;  
            System.out.println(c);  
        }  
        catch (Exception e)  
        {  
            System.out.println(e);  
        }  
        System.out.println("Hello");  
    }  
}
```

// output:

java.lang Arithmetic Exception : / by zero
Hello

Q23) Write a program in Java to implement Thread Class?

Ans) code:-

class Test extends Thread

{

public void run()

{

System.out.println("Thread Test");

}


```

public static void main (String[] args)
{
    Test t = new Test ();
    t.start ();
}
}

```

// output:
Thread task.

Q24) Write a program in Java to implement Runnable Interface.
Ans) code:-

```

class Test implements Runnable
{
    public void run ()
    {
        System.out.println ("Thread task");
    }
}

```

```

public static void main (String[] args)
{
    Test t = new Test ();
    Thread th = new Thread (t);
    th.start ();
}
}

```

// output:
Thread task.

Q25) Explain the difference b/w checked and unchecked exceptions in Java with examples.

Ans) checked exceptions
i) checked exceptions happen at compile time when the source code is transformed into an executable code.

ii) The checked exception is checked by the compiler.

iii) checked exceptions can be created manually.

iv) It is a sub-class of the exception class.

v) Checked exceptions is also called Compile Time exceptions.

JVM requires the exception to be caught or handled.

Ex: IOException, FileNotFoundException.

Unchecked exceptions
unchecked exceptions happen at runtime when the executable program starts running.

These types of exceptions are not checked by the compiler.

They can also be created manually.

These are not a part of the exception class since they are runtime exception.

Unchecked exceptions is also called Run-time Exception.

JVM does not require the exception to be caught or handled.

Ex: NullPointerException, ArithmeticException.

Q26) Write a program in Java to implement getPriority in multi-Threading.

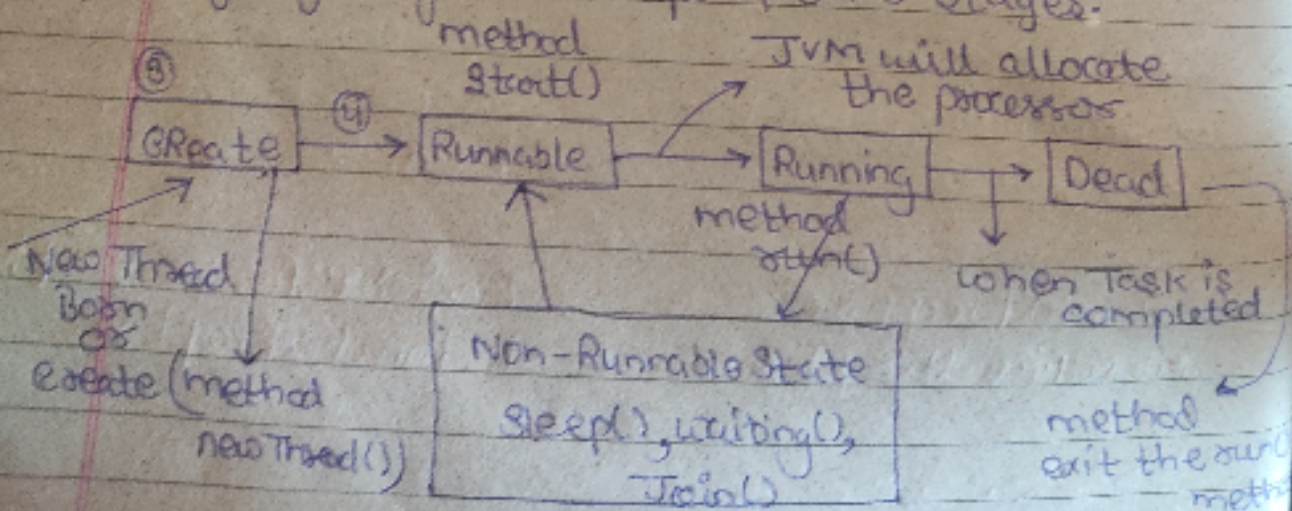
Ans) Code

```
class Test extends Thread
{
    public void run()
    {
        System.out.println("child thread");
    }
    public static void main (String[] args)
    {
        sop (Thread.currentThread().getPriority());
        Test t = new Test();
        t.start();
    }
}
```

// output :

Q27) Describe the life cycle of a thread in Java with method associated with each stage.

Ans) Life cycle of thread depends on 5 stages:



Q28) write a program in Java to implement multithreading in Java.

Ans) Code :-
multithreading using the Runnable Interface

```
class Test implements Runnable
{
    public void run()
    {
        System.out.println("Thread task");
    }
    public static void main (String[] args)
    {
        Test t = new Test();
        Thread th = new Thread(t);
        th.start();
    }
}
```

// output:
Thread task.

Q29) write a program in Java to implement Multiple task from multiple Thread.

Ans) Code :-

```
class Test1 extends Thread
{
    public void run()
    {
        System.out.println("Volume 1");
    }
}
```



```

class Test2 extends Thread
{
    public void run()
    {
        System.out.println("Hello programmer");
    }
}

class Buzo
{
    public static void main(String[] args)
    {
        Test t1 = new Test1();
        t1.start();
        Test t2 = new Test2();
        t2.start();
    }
}

```

//output:
 Volume1
 Hello programmer.

Q30) Write a java programme to implement setPriority() in multi-Threading.

Section - C

Q31) Explain the use of throw keyword and give full Implementation.

Ans) The throw keyword in java is used to explicitly throw an exception. When an exception is thrown using throw, the normal flow of the program is disrupted, and the exception is passed up the call stack, looking for an appropriate exception handler (i.e., a catch block).

Throw always use user-defined/customized exception. Throw keyword only works on (Unchecked Exception).

Syntax :-

throw new Exception className ("String message");

Code :-

```
class YoungerAgeException extends RuntimeException
{
    YoungerAgeException (String message)
    {
        super (message);
    }
}
```



```

class voting
{
    public static void main(String[] args)
    {
        int age = 18;
        try
        {
            if (age < 18)
            {
                YoungerAgeException("Not eligible for voting");
            }
        }
        else
        {
            System.out.println("vote successfully");
        }
    }
}
catch (YoungerAgeException e)
{
    e.printStackTrace();
}
}

```

// output :

Exception in thread "main" YoungerAgeException:
 Not eligible for voting at voting.main (voting.java:6)

Q32)

Ans)

Write a program in Java to implement throws.
 Code 3-

```

import java.io.*;
import java.io.*;
class ReadAndWrite
{

```

Q33)

Ans)


```

void readfile() throws FileNotFoundException
{
    FileInputStream fis = new FileInputStream(
        "d:\\abc.txt");
}
}
class Test
{
    public static void main(String[] args)
    {
        ReadAndWrite rwo = new ReadAndWrite();
        try
        {
            rwo.readfile();
        }
        catch (FileNotFoundException e)
        {
            System.out.println(e);
        }
    }
}

```

Q.6) @33) Write a java program to simulate a scenario with multiple threads that require synchronization and demonstrate the use of synchronized blocks.

Ans) Code :-

```

class Book TheatreSeats
{
    int total_seats = 10;
}

```


synchronized void BookSeats (int seats)

if (total_seats >= 10)

System.out.println("Seats booked Successfully");

total_seats = total_seats - seats;

System.out.println("Seats left:" + total_seats);

else

System.out.println("Sorry your seat cannot
be booked");

System.out.println("Seats left:" + total_seats);

class MovieBookApp extends Thread

{
static Book TheatreSeats b;

int seats;

public void run()

{
b.BookSeats(seats);
}

public static void main (String[] args)

{
b = new Book TheatreSeats();

MovieBookApp Amit = new

MovieBookApp();

Amit.seats = 7;

Amit.start();


```

MovieBookApp Ajay = new MovieBookApp();
Ajay.seats = 6;
Ajay.start();
}

```

```

//output:
Seats booked Successfully.
Seats left: 3.
Sorry, your seat cannot be booked.
Seats left: 3.

```

The use of synchronized in Java when multiple thread access and modify a shared variable, synchronized is essential to avoid inconsistencies.

Q35) Explain Four Different cases of multiple Threading with Implementation.

Q36) b) What are the various possible combination of try-catch-finally blocks.

Ans)

Case I:-

```
try
```

```
{
```

```
}
```

```
catch
```

```
{
```

```
}
```

```
finally
```

```
{
```

```
}
```

Single reproduce does not execute

Case II:-

```
try
```

```
{
```

```
}
```

```
catch
```

```
{
```

```
}
```

```
finally
```

```
{
```

```
}
```

✓ SOP(),

```
}
```

These together

will execute well

Case III:-

```
try
```

```
{
```

```
}
```

```
finally
```

```
{
```

```
    sop();
```

```
}
```

without

catch will

execute

Case IV:-

```
try
```

```
{
```

```
try
```

```
{
```

```
    [ ]
```

```
catch
```

```
{
```

```
}
```

```
}
```

```
}
```

```
catch
```

```
{
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

Case V:-

```
try
```

```
{
```

```
}
```

```
catch
```

```
{
```

```
}
```

```
try
```

```
{
```

```
}
```

```
catch
```

```
{
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

Case VI:-

```
try
```

```
{
```

```
}
```

```
catch (Exception e)
```

```
{
```

```
}
```

```
catch (ArithmeticException e)
```

```
{
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

will give error.

Case 80118

1.2

1

1

catch (RuntimeException e)

1

1

catch (IOException e)

1

1

Q1) Flowchart of Exception Hierarchy in details.